REDROB HACKATHON

Intelligent Candidate Discovery & Ranking

Ranking 100,000 candidates the way a great recruiter would -- not by counting keywords, but by understanding career trajectory, evidence of skill, and platform behavior together.

100,000	~55 sec	0%	2.8 GB
candidates ranked	full-pool runtime	honeypots in top 100	peak memory

THE PROBLEM

Keyword filters can't see what actually matters

The job description states this directly: the goal is not finding candidates whose skills section contains the most AI keywords.

Naive baseline ranker's #1 pick

HR Manager

6.1 years experience | 9 “AI core” skill tags | 0.76 recruiter response rate

Ranked #1 by the bundled sample_submission.csv, which sorts purely by AI-keyword count and response rate. An HR Manager's skill tags do not make them a search/ranking engineer.

A real “keyword-stuffer” in the data

Backend/Data Engineer at Mindtree

Skills list: NLP, Fine-tuning LLMs, LoRA, GANs, Milvus, Image Classification…

Their own profile summary: “Most of my career has been data engineering, with some adjacent ML exposure… building competence on the ML side.” Every trending keyword, but not the role.

STEP ONE -- EXPLORE BEFORE YOU BUILD

The pool draws from a closed, enumerable universe

Before writing any scoring logic, we enumerated every distinct value the 100K-candidate pool actually contains.

63

unique companies

Hand-classified by type: AI-native, big-tech, Indian product company, SaaS, consulting, or fictional placeholder

48

unique job titles

Hand-tiered 0-5 by relevance to the role, e.g. “Search Engineer” = 5, “HR Manager” = 0

133

unique skills

Weighted 0-1 by genuine relevance to retrieval/ranking, not just “AI-sounding”

Why this matters: with a universe this small, hand-built lookup tables beat fuzzy matching. We can answer “is Infosys a consulting company?” directly and correctly, every time -- not approximate it with an embedding model.

ARCHITECTURE

Three multiplicative layers, fully offline

$$score = disqualifier \times behavioral \times core\ fit$$

Core Fit Score

7-component weighted blend: title tier, semantic similarity, trust-weighted skills, experience band, company quality, location, education.

range: 0 - 1

×

Disqualifier Multiplier

Encodes the JD's explicit “do NOT want” list: pure-research-only, consulting-only, CV/speech-only, framework-only AI exposure, title-hopping.

range: 0.25 - 1.0

×

Behavioral Multiplier

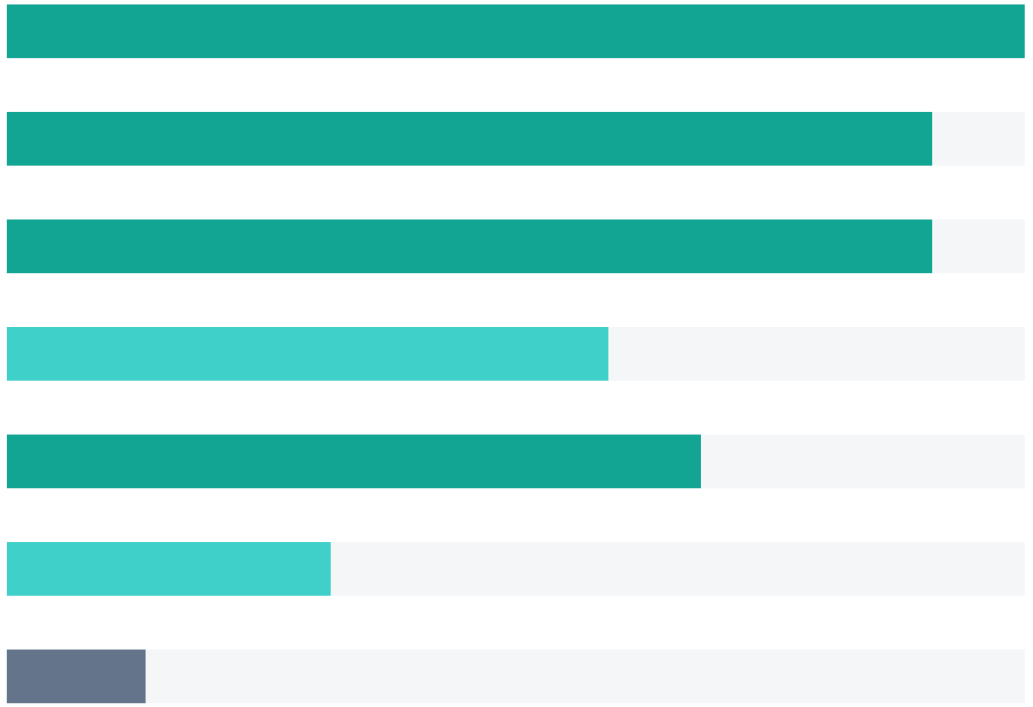
Down-weights inactive, unresponsive, or unreachable candidates -- calibrated against the pool's own signal percentiles.

range: 0.45 - 1.2

CORE FIT SCORE

Seven components, weighted by what the JD actually asks for

Title relevance	0.22
Career-history semantic similarity	0.20
Trust-weighted skill match	0.20
Experience-band fit	0.13
Company quality / AI-relevance	0.15
Location fit	0.07
Education tier	0.03



Weights are hand-set from reading the JD's stated priorities -- see docs/approach.md for the full per-component rationale and what we'd change with ground-truth labels.

DISQUALIFIER MULTIPLIER

The JD's “do NOT want” section, encoded directly

The JD is unusually explicit about disqualifiers. We treat each as a soft multiplier, not a hard zero -- the JD itself hedges every one (“we will probably not move forward”).

Pure research-only career	× 0.25	32 candidates	Entire history in research titles, no production-deployment role
100% consulting-only career	× 0.35	9,745 candidates	Every role at TCS/Infosys/Wipro-type firms, no product company exposure
Framework-only AI exposure	× 0.55	2,424 candidates	LangChain / RAG / Prompt Engineering only -- no retrieval or ranking depth
CV/speech-only specialist	× 0.50	4,417 candidates	Computer vision or speech skills with no NLP/IR exposure
Sustained title-hopping	× 0.70	47 candidates	Every role ≤18mo with 4+ jobs -- no anchor of depth anywhere

HONEYPOT DETECTION

Two reliable signals beat five plausible-sounding ones

We tested every “subtly impossible” pattern we could think of against the full 100K pool, and kept only what was rare enough to actually be a trap.

KEPT -- genuinely rare

“Expert” skill, 0 months used

21 candidates

Impossible by definition

Career-history months ≫ stated years of experience

24 candidates

No overlapping-job support in the schema

Total excluded before scoring: 45 | Measured honeypot rate in final top 100: 0%

REJECTED -- too common to be the trap

Salary min > max

18,865 candidates (~19% of pool)

Education ends after first job starts

19,499 candidates (~19% of pool)

Skill duration ≫ total experience

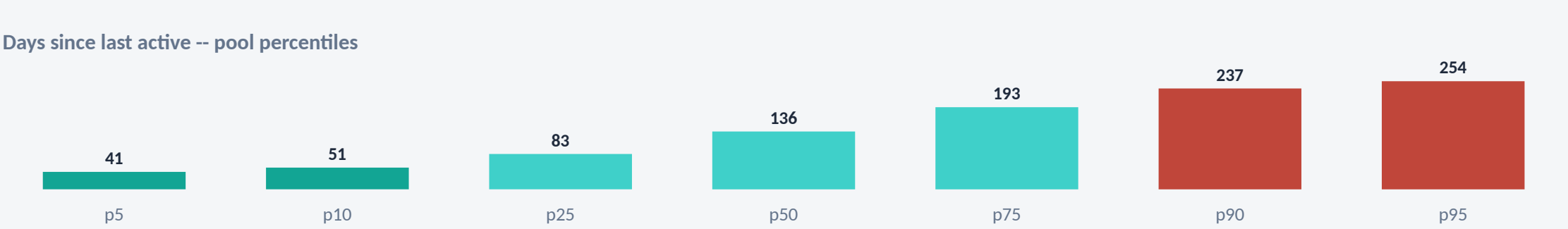
2,821 candidates (~2.8% of pool)

These each fired on a sixth to a fifth of the entire pool -- far too common to be the ~80 intended honeypots. Just noise in the synthetic data generator.

BEHAVIORAL MULTIPLIER

Calibrated against the pool's real distribution, not guesses

Our first version assumed “active within 14 days” was meaningful. Checking the actual data told a different story:



Finding: nobody in the entire 100K pool has been active in the last ~30 days. The most recent candidate is ~32 days inactive. A flat “≤14 days = recent” bonus was dead code.

Fix: every threshold (recency, response rate, interview completion, notice period) is now set against the pool's own p10/p25/p50/p75/p90 -- not an assumed absolute value. Notice-period median, for example, is 90 days, not 30.

SEMANTIC SIMILARITY

TF-IDF + LSA over career history, not a transformer encoder

The JD explicitly rewards latency-quality tradeoff thinking: an LLM call per candidate “will not fit the 5-minute CPU budget.” We made the same call one level down.

Compute budget

A transformer forward-pass over 100K career-history texts on CPU eats a meaningful slice of the 5-minute budget by itself, before any scoring logic runs.

Right regime for the method

Resume and JD text share a fairly constrained professional vocabulary -- exactly where TF-IDF + LSA is competitive relative to its cost.

Inspectability

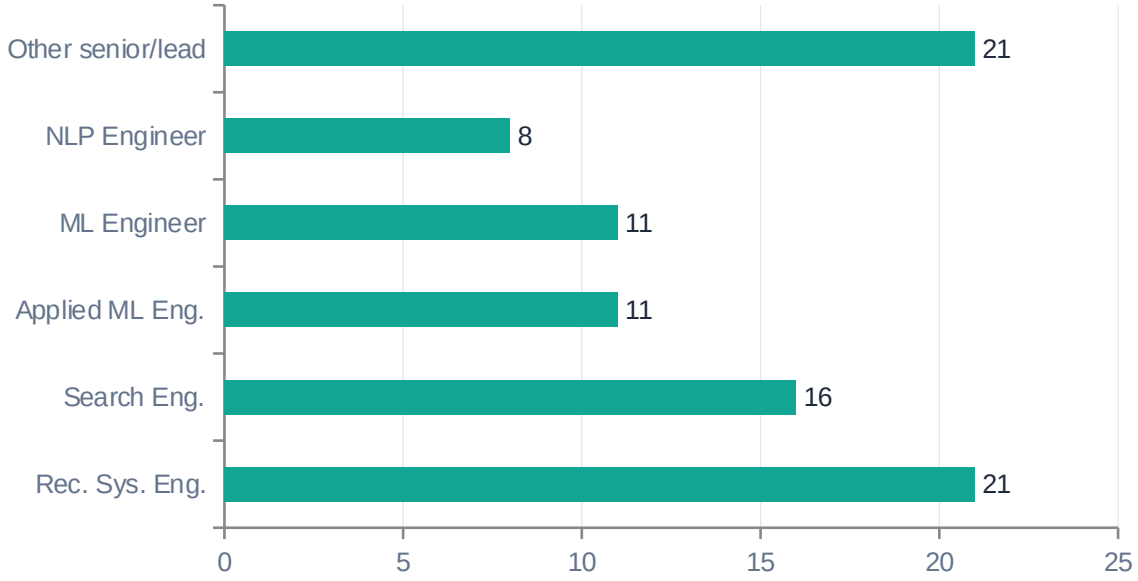
Every similarity score traces back to specific shared terms -- useful for the Stage 4 review and the Stage 5 interview alike.

Future work: with transformer/network access, the natural next step is a small local sentence-encoder behind the same semantic.py interface -- swapped in without touching the rest of the pipeline.

RESULTS

What the final top-100 shortlist actually looks like

Title distribution in the top 100



91 / 100
are India-based
Matches the JD's strong geography preference

0%
honeypot rate in top 100
Well under the 10% disqualification threshold

6.7 yrs
mean experience
Centered in the JD's 5-9 year band

~55 sec
full 100K-pool runtime
Measured, not estimated -- well under the 5-min budget

QUALITY CONTROL

We caught our own templated reasoning before submitting

Stage 4 review explicitly samples 10 rows and checks: are the reasonings substantively different from each other? Our first draft failed this by construction.

BEFORE -- every top-100 row, near-identical

"title closely matches the role; career history closely matches the JD's retrieval/ranking focus; well-evidenced skills in..."

"title closely matches the role; career history closely matches the JD's retrieval/ranking focus; strong, endorsement-backed..."

Why: every top-100 candidate scores well on title and semantic similarity by definition -- so a reasoning template keyed on "score is high" produces near-identical text everywhere.

AFTER -- specific to each candidate

"...as Staff ML Engineer at Niramai (44mo) their role covers Pinecone; strong, endorsement-backed depth in Learning to Rank, Elasticsearch, BM25."

"...but based in London, UK -- outside the JD's India preference, no visa sponsorship." An honest concern, surfaced even at a strong rank.

Fix: anchor on each candidate's single most-relevant past role + the specific keywords in its description, plus their best-evidenced skills, plus at least one honest concern when one exists.

MEASURED, NOT ASSUMED

Compute constraints -- verified against the full 100K pool

Wall-clock runtime	≤ 5 minutes	**~50-65 seconds**
Peak memory	≤ 16 GB	**~2.8 GB**
GPU usage	None permitted	**Confirmed CPU-only**
Network calls during ranking	None permitted	**Zero -- confirmed in source**

```
python src/rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

One command, end to end, from the raw candidate pool to the validated top-100 CSV. No manual steps, no pre-computation, no hidden state.